

MODULE 5

Introduction to Spark

MODULE-5

Spark and Big Data Analytics: Spark, Introduction to Data Analysis with Spark.

Text, Web Content and Link Analytics: Introduction, Text Mining, Web Mining, Web Content and Web Usage Analytics, Page Rank, Structure of Web and Analyzing a Web Graph.

TB2: Ch5: 5.2,5.3, Ch 9: 9.1-9.4

Pig or Hive are high-level scripting languages that are used with the Apache Hadoop.

They have SQL like commands for queries.

The commands before executing, translate to **Map and Reduce parallel-tasks**, process the **queries**, **built-in functions**, **aggregation operations** and **User Defined Functions (UDFs)**.

They enable **ease in programming** for these functions.

They run **ETL processes using Big Data Store**.

Pig and Hive programs use **complex data types and operations**.

Applications such as **data analytics, stream analytics and graph analytics**, and **machine learning** require the following:

In-memory processing:

- fast

- less time

- facilitates real-time processing and streaming data analysis.

Application tasks processing Framework:

- uses HDFS as well Hadoop compatible data sources, such as HBase, Cassandra, Ceph, cloud-based Objects Store Service or Amazon S3.

- The tasks require support which facilitates running the Hive, Pig, and other Hadoop ecosystem tools in Java, Python, R and Scala.

- APIs using Python shell and Scala shell facilitate the interactive running of the applications.

Open Source Analytics Tools

1. **R and its library** provide various statistical analysis functions. R now analyses large data sets also since R integrates with Big Data platforms, such as Spark.
2. **Python** is a widely used language due to its analysis and statistics libraries, such as numpy, scipy, scikit-learn, pandas, StatsModel.
3. **Storm** is for real-time continuous data streams.
4. **Pig is a data flow language with SQL like** operations and uses UDFs. Pig enables easy coding compared to MapReduce for the complex tasks.
5. **Hive is for creation of data warehouse**, integration of databases and applications, and uses SQL like scripts and UDFs. Coding is easy compared to MapReduce.

The most popular open source analytics tools are **Apache Spark, Python, R, Apache Pig and Hive**

- **Spark** is for high volume unstructured data.
- **Spark** seamlessly integrates with Spark 4. which uses the structured data,
- **Spark Streaming** is for streaming data,.
- **Spark Graphx** for graph databases, Spark MLlib for machine-learning library, and Spark Arrow for columnar in memory analytics.
- **Spark** provides easy programmability with inclusion of APIs for programmers to develop applications in Python, R, Java or Scala.

SPARK

Apache Spark is a fast and general compute engine.

Apache Spark™ powers the analytics applications up to **100 times faster**.

It supports **HDFS compatible data**.

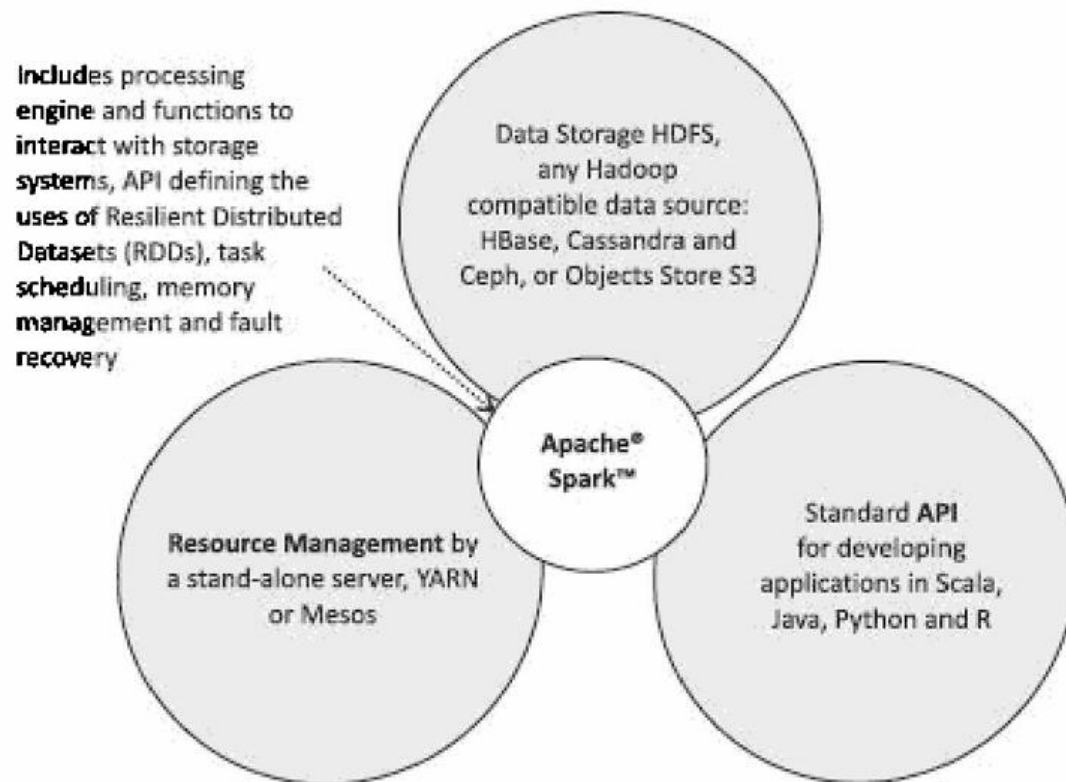
Spark has a **simple and expressive programming model**.

Spark runs on both **Windows and UNIX-like systems**, such as Linux and Mac OS.

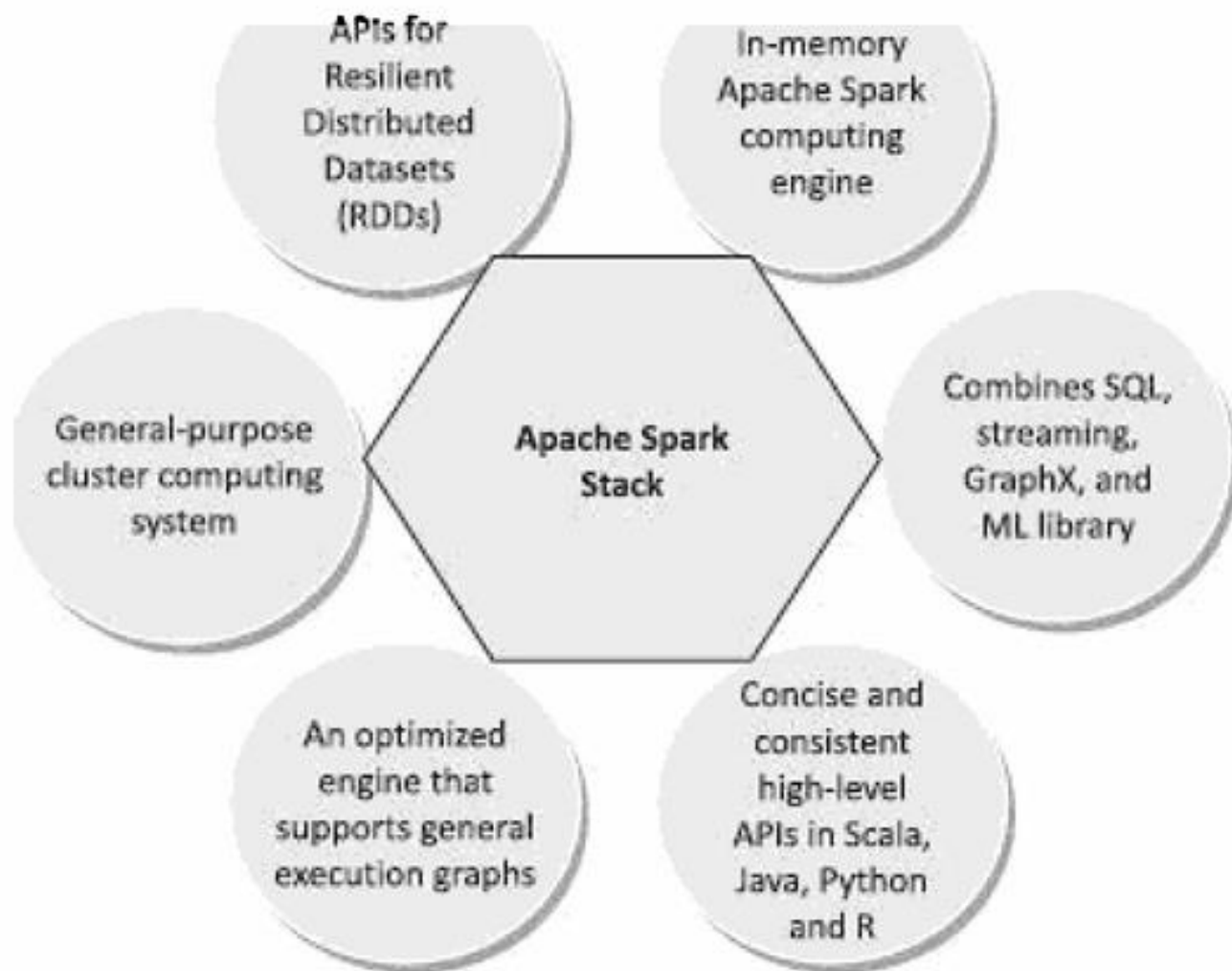
Java is essential for running Spark applications.

The multiple languages, Python and Scala shells provide great **ease in programming** for complex analytics, machine learning and other solutions.

Introduction to Big Data Tool- Spark



Spark HDFS file system
Spark Standard API
Spark resource management



SPARK SQL



STREAMING



MLLIB



GRAPHX



CORE SPARK

YARN

MESOS

KUBERNETES

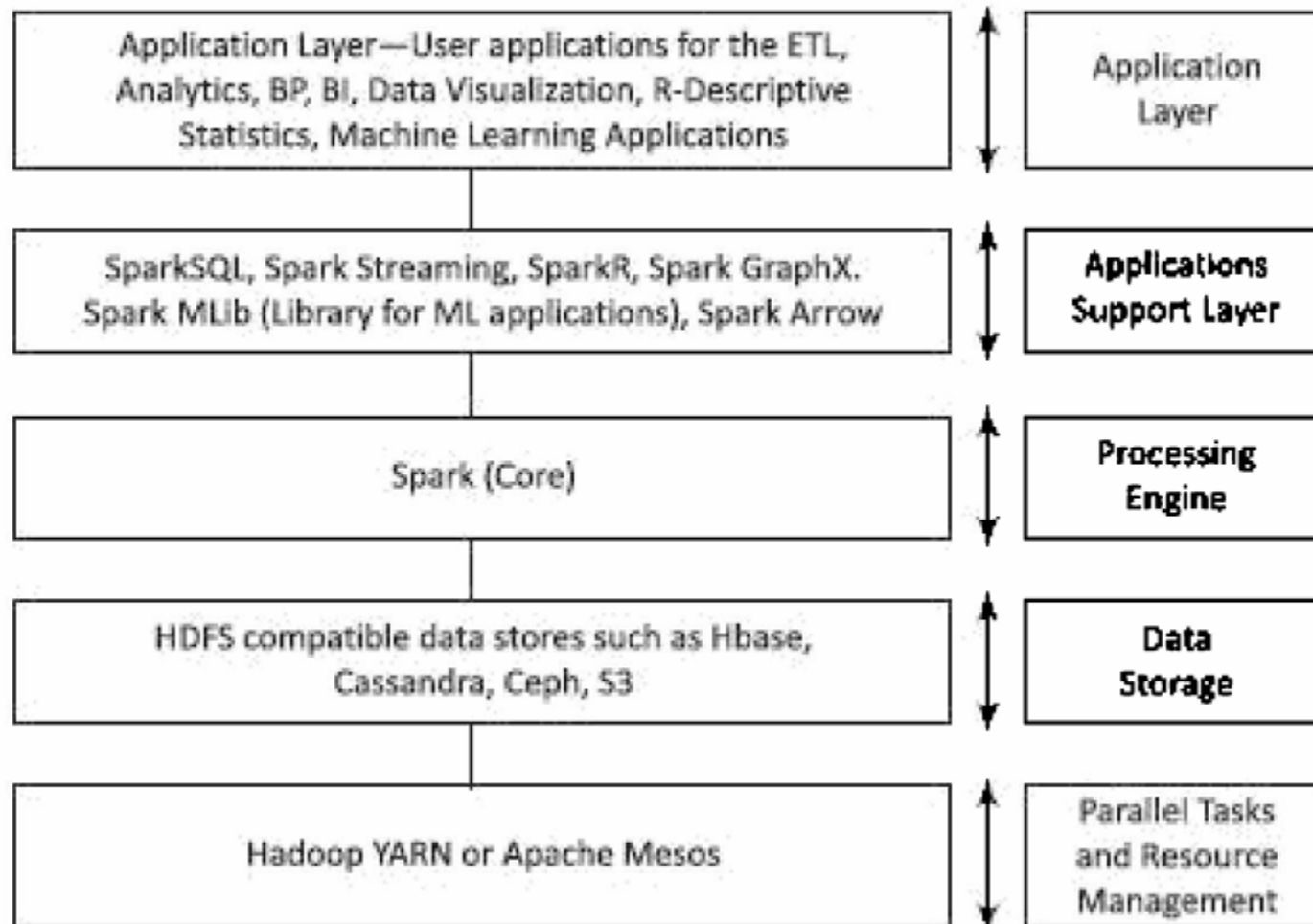
DOCKER
SWARM

the features of Spark:

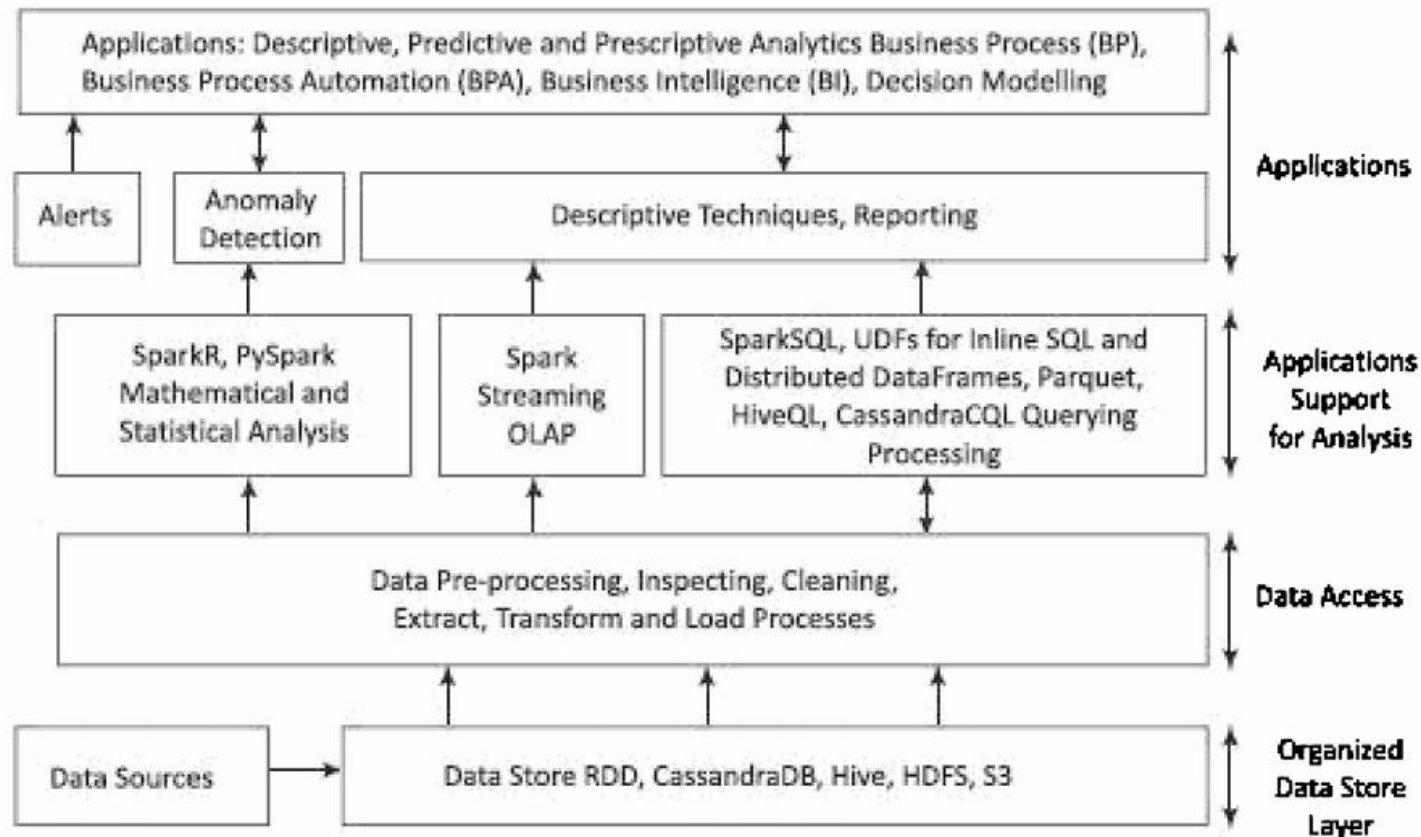
1. Spark provisions for creating applications that use the complex data. In-memory Apache Spark computing engine enables up to **100 times** performance with respect to Hadoop.
2. Execution engine uses both in-memory and on-disk computing. Intermediate results save in-memory and spill over to disk.
3. Data uploading from an Object Store for immediate use as a Spark object instance. Spark service interface sets up the Object Store.
4. Provides high performance when an application accesses memory cache from the disk.
5. Contains API to define Resilient Distributed Datasets (RDDs).
 - RDD is the core concept in Spark framework.
 - RDD represents a collection of Object Stores distributed across many computer nodes for parallel processing.
 - Spark stores data in RDD on different partitions.

6. Processes any kind of data, namely structured, semi-structured or unstructured data arriving from various sources.
7. Supports many new functions in addition to Map and Reduce functions.
8. Optimizes data processing performance by slowing the evaluation of Big Data queries.
9. Provides concise and consistent APIs in Scala, Java and Python. Spark codes are in Scala and run on JVM environment.
10. Supports Scala, Java, Python, Clojure and R languages.
11. Provides powerful tool to analyze data interactively using shell which is available in either Scala (which runs on the Java VM and is thus a good way to use existing Java libraries) or Python. The tool also provides for learning the usages of API.

Spark Software Stack



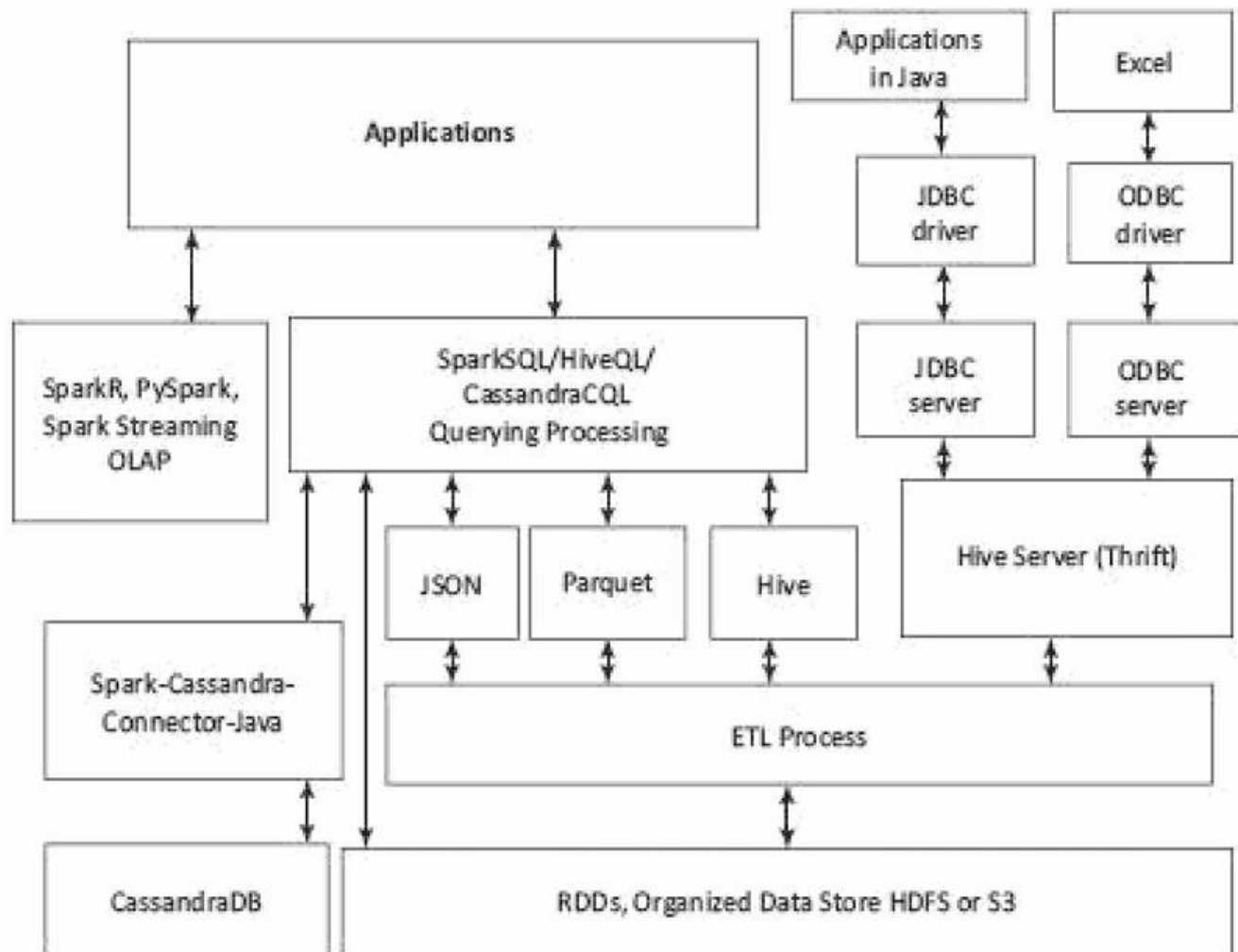
Introduction to data analysis with Spark



- Steps between acquisition of data from different sources and its applications

Following are the steps for analyzing the data:

1. **Data Storage**: Store of data from the multiple sources after acquisition. The Big Data storage may be in HDFS compatible files, Cassandra, Hive, HDFS or S3.
2. **Data pre-processing**: This step requires:
 - (a) dropping out of range, inconsistent and outlier values,
 - (b) filtering unreliable, irrelevant and redundant information,
 - (c) data cleaning, editing, reduction and/or wrangling,
 - (d) data-validation, transformation or transcoding.
3. **Extract, transform and Load (ETL)**) for the analysis
4. Mathematical and **statistical analysis of the data obtained** after querying relevant data needing the analysis, or OLAP,
5. Applications of analyzed data, for example, **descriptive, predictive and prescriptive analytics, business processes (BPs), business process automation (BPA), business intelligence (BI), decision modelling and knowledge discovery.**



Connectivity between the applications and Spark SQL

1

Assume a table `toyPuzzleTypeCostTbl` with four columns. Figure 5.6 shows the sample table `toyPuzzleTypeCostTbl`. The columns are puzzle type, puzzle code, number of puzzle pieces and puzzle cost. DataFrame1 named `toyPuzzleTypeCodes` consists of columns 1 and 2. DataFrame2 named `toyPuzzleCodesCost` consists of Columns 2 and 4. The table consists of multiple rows in each row group. The dashed lines point to the columns in the two data frames.

toyPuzzleTypeCostTbl				
	puzzleType	puzzleCode	puzzlePieces	puzzleCost
Row Group1	puzzle_ Garden	10725	100	1.35
	puzzle_ Garden	10825	200	1.35
	puzzle_ Garden	10975	400	1.35
	...	...	...	...
Row Group2	puzzle_ Jungle	31047	300	2.85
	puzzle_ Jungle	31047	300	2.85
	...	...	...	...
Row Group3	puzzle_ School	81409	800	0.90
	...	...	...	...
	puzzle_ Forest	...	...	...

DataFrame toyPuzzleTypeCodes

Columns 1 and 2

DataFrame toyPuzzleCodesCost

Columns 2 and 4

Figure 5.6 Sample table toyPuzzleTypeCostTbl rows, row groups and DataFrames

- (i) Create a sqlContext from a given SparkContext 'sc'.
- (ii) Create a four columns DataFrame using the Parquet file named "toyPuzzleTypeCostTbl".
- (iii) How will a DataFrame create using a JSON file format file "toyPuzzleTypeCostTbl"?
Create two DataFrames using Java and SqlContext, one with columns for puzzle type and puzzle code, and the other for puzzle code and cost.
- (iv) How will two DataFrames join using puzzle code as a join key to create a DataFrame of three columns, 1, 2 and 4?

SOLUTION

- (i) The following statement creates sqlContext from Spark Context sc:

```
SqlContext      sqlContext      =      new  
org.apache.spark.sql.SQLContext(sc)
```

- (ii) The following statement creates a DataFrame named toyTypeCost:

```
DataFrame      toyTypeCost      =  
sqlContext.parquetFile("toyPuzzleTypeCostTbl")
```

[DataFrame created will have four columns.]

- (iii) DataFrame creates using Load() method at the sqlContext.

```
# To display the contents of Table:
"toyPuzzleTypeCostTbl"

spark.sql ("SELECT * FROM toyPuzzleTypeCostTbl ")

# To create DataFrame toyPuzzleTypeCostTbl

DataFrame toyPuzzleTypeCostTbl = sqlContext.Load
("toyPuzzleTypeCostTbl", "json")
```

The following statements create two DataFrames using DataFrame toyPuzzleTypeCostTbl. One frame of two columns, 1 and 2, puzzle type and puzzle code:

```
DataFrame toyTypeCodes = toyPuzzleTypeCostTbl.
select(toyPuzzleTypeCostTbl ['puzzleType'],
toyPuzzleTypeCostTbl ['puzzleCode'])
```

Second frame of two columns, 2 and 4, puzzle code and puzzle cost

```
DataFrame toyCodesCost =
toyPuzzleTypeCostTbl.select(toyPuzzleTypeCostTbl
['puzzleCode'], toyPuzzleTypeCostTbl ['puzzleCost'])
```

- (iv) DataFrame1 (toyTypeCodes) has two columns, 1 and 2, as *puzzleType* and *puzzleCode*, respectively. The frame joins dataframe2 (toyCodesCost) column 4 as *puzzleCost* and resultant is a joined new-dataFrame **toyTypeCodesCost** consisting of columns 1, 2 and 4. Join key is *puzzleCode*. Following statements use join() method and joining key.

Format of the statement is

```
dataFrame. join (dataframe1, dataframe2.col ("JoinKey") = dataframeNew  
("JoinKey")
```

and the statement is

```
dataFrame.join(toyTypeCodes,  
toyCodesCost.col("puzzleCode").  
equalTo (toyTypeCodesCost ("puzzleCode")))
```

CarShowroomsCumulativeYearlySales

Car ShowroomID (csID)	Date (DT) mmddyy	<i>Jagaur Land Rover</i> Sale (JLRDS)	<i>Hexa</i> Sale (HDS)	<i>Zest</i> Sales (ZDS)	<i>Nexon</i> Sale (NDS)	<i>Safari Storme</i> Sale (SSDS)
220	----	--	---	----	---	---
10	121217	49	34	164	115	38
122	121217	40	141	123	37	88
-	-	-	-	-	-	-
-	-	-	-	-	-	-

- (i) Find the **Jagaur Land Rover** annual sale figure over all showrooms.
- (ii) Find the showroom ID and Land Rover sales figure for the showroom giving *maximum Jaguar Land Rover sales*.

SOLUTION

- (i) The following query returns the sum of the **Jagaur Land Rover** annual sale in column 3 of the table:

```
SELECT sum (JLRDS) FROM CarShowroomsCumulativeYearlySales
```

- (ii) The following nested query statement returns the csID and maximum annual sale value for Land Rover:

```
SELECT csID, saleJL (max (map (csID, ID, JLRDS, saleJL)))  
FROM CarShowroomsCumulativeYearlySales
```

- (i) How is HiveContext and Spark SQL used?
- (ii) How does PySpark use a row object?
- (iii) Assume a JSON file, *toyTypeProductTbl* of row objects. Assume the use of a row object *toyPuzzleProduct* for query (Table 4.13). How do a file load and query sent to the file?

	ProductCategory	ProductId	ProductName
Row Object1	Toy_Airplane	10725	Lost Temple
Row Object2	Toy_Airplane	****	*****
Row Object3	****	*****	*****

- (i) Import Spark SQL using the following statement:

```
Import Spark SQL
```

- (ii) Then use the following command for importing a row object, *toyPuzzleTypeCost* :

```
# Now use the variable named hiveCtx using Hive Context  
from
```

```
# sc statement using statement
```

```
hiveCtx = HiveContext (sc)
```

```
from pyspark.sql import HiveContext, toyPuzzleTypeCost
```

- (iii) A file loads using *hiveCtx* and input loads at the file


```
input = hiveCtx.jsonFile(toyTypeProductTbl)
```

```
input.registerTempTable ("toyPuzzleProduct")
```

The queries raised for finding Product_ID_Name using SELECT command.

```
Product_ID_Name = hiveCtx.sql ("SELECT ID, name FROM  
toyPuzzleProduct ORDER BY ProductCategory")
```

Add 10% cost in each value of item cost in a group of rows. Use GVUDF to define a DataFrame costTenPercetPlus GVUDF.

SOLUTION

Following are the Python statements from pyspark.sql.functions import pandas_udf:

```
from pyspark.sql.functions import pandas_udf
```

[Use pandas_udf to define a grouped vectorized udf.]

```
@pandas_udf(df.schema)
```

```
# Input/output are both a pandas.DataFrame
```

```
def costTenPercetPlus (pdf):
```

```
    return pdf.assign(v=add(pdf.v + 0.1*pdf.v))
```

```
df.groupby('id').apply(costTenPercetPlus)
```

Data Analysis Operations

Examples of operations required in the above analysis are given below:

1. Filtering single and multiple columns
2. Creating a top-ten list with values or percentages
3. Setting up sub-totals
4. Creating multiple-field criteria filters
5. Creating unique lists from repeating field data
6. Finding duplicate data with specialized arrays, and using the remove duplicates command and removing outliers
7. Multiple key sorting
8. Counting the number of unique items in a list
9. Using SUMIF and COUNTIF functions
10. Working with database functions, such as DSUM and DMAX
11. Converting lists to tables.

How will you remove outliers in values in a column?

SOLUTION

Transform RDD string or other to numeric data so that statistical methods compute and remove those who have larger distanceNumerics.

```
distanceNumerics = distances.map (PySpark string: float  
(string))
```

```
stats = distanceNumerics.stats()
```

[stats() means a statistical function, such as mean(), stdev()]

```
statdev = std.stdev()
```

```
mean = stats.mean()
```

```
reasonableDistances = distanceNumerics.filter (PySpark
```

```
values: maths.fabs (values-mean) < 3 ×stdev)
```

[Assume that distances that are reasonable are less than three times the standard deviation. Distance means difference with respect to mean or peak value.]

```
print reasonableDistances.collect()
```

[Print the values within reasonable distances.]

Text, Web Content and Link Analytics

Introduction

- Text analysis, sometimes called text analytics, refers to the representation, processing, and modeling of textual data to derive useful insights.
- An important component of text analysis is text mining, the process of discovering relationships and interesting patterns in large text collection
- Text analysis suffers from the curse of high dimensionality.
- Another major challenge with text analysis is that most of the time the text is not structured.

Example Data Sources and Formats for Text Analysis

Data Source	Data Format	Data Structure Type
News articles	TXT, HTML, or Scanned PDF	Unstructured
Literature	TXT, DOC, HTML, or PDF	Unstructured
E-mail	TXT, MSG, or EML	Unstructured
Web pages	HTML	Semi-structured
Server logs	LOG or TXT	Semi-structured or Quasi-structured
Social network API firehoses	XML, JSON, or RSS	Semi-structured
Call center transcripts	TXT	Unstructured

TEXT MINING

Today, large amounts of textual data is generated in computing applications. Text stream arriving continuously over time generates text data. For example, news articles, news reports, online comments on news, online traffic reports, corporate reports, web searches, and contents at social media discussion-forums (such as LinkedIn, Twitter and Facebook), short messages on phones, chat messages, transcripts of phone conversations, blogs and e-mails.

Text Mining

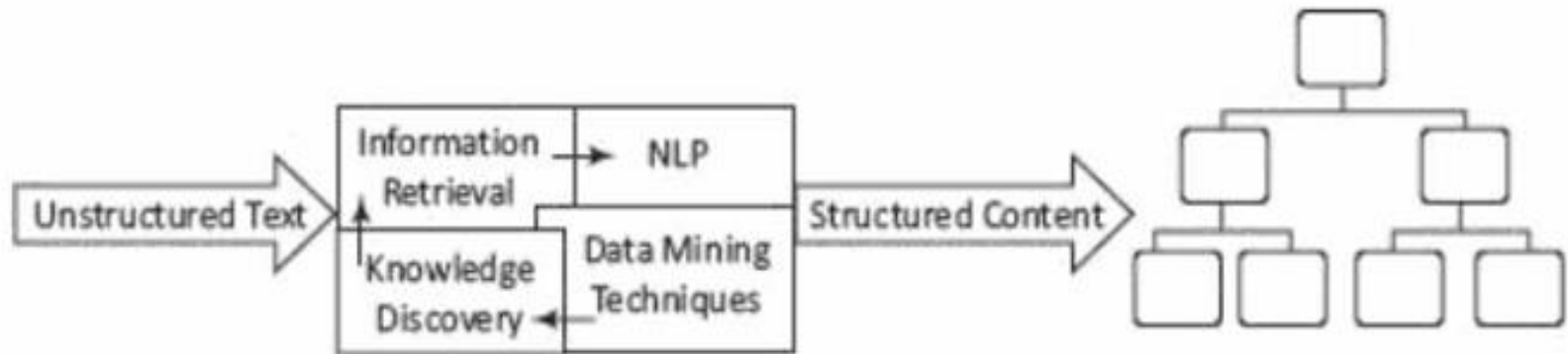
Four definitions are:

1. "Text mining refers to the process of deriving high-quality information from text." (Wikipedia)
2. "Text mining is the process of discovering and extracting knowledge from unstructured data." (National Center of Text Mining —The University of Manchester¹)
3. "Text mining is the process of analyzing collections of textual contents in order to capture key concepts themes, uncover hidden relationships, and discover the trends without requiring that you know the precise words or terms that authors have used to express those concepts." (IBM²)
4. "Text mining is a technique which helps in revealing the patterns and relationships in large volumes of textual content that are not visible to the naked eye, leading to new business opportunities and improvements in processes." (Amazon BigData Official Blog³)

Applications of text mining in business domains are predicting stock movements from analysis of company results, decision making for product and innovations developed at the company and contextual advertising. Some other applications are (i) mail filtering (spam), (ii) drug action reports (iii) fraud detection (iv) knowledge management, and (iv) social media data analysis.

The applications provide innovative and insightful results. The results when combined with other data sources, find the answers to the following:

- (i) Two terms which occur together
- (ii) Information linkage with another information
- (iii) Different categories that can be created from extracted information
- (iv) Prediction of information or categories.



9.2.1.2 Areas and Applications of Text Mining

Natural Language Processing (NLP) is a technique for analyzing, understanding and deriving meaning from human language. NLP involves the computer's understanding and manipulation of human language. NLP algorithms are typically based on ML algorithms. They automatically learn the rules. First, they analyze set of examples from a large collection of sentences in a book. Then, they make the statistical inferences.

Information Retrieval (IR) is a process of searching and retrieving a subset of documents from the abundant collection of documents. IR can also be defined as extraction of information required by a user. IR is an area derived fundamentally from database technology. One of the most popular applications of IR is searching the information on the web. Search engines provide IR using various advance techniques. For example, the crawler program is capable of retrieving information from a wide variety of data sources. Search methods use metadata or full-text indexing.

Information Extraction (IE) is a process in which the software extracts structured information from unstructured and/or semi-structured documents. IE finds the relationship within text or desired contents from text. IE ideally derives from machine learning, more specifically from the NLP domain. Content extraction from the images, audio or video is an example of information extraction.

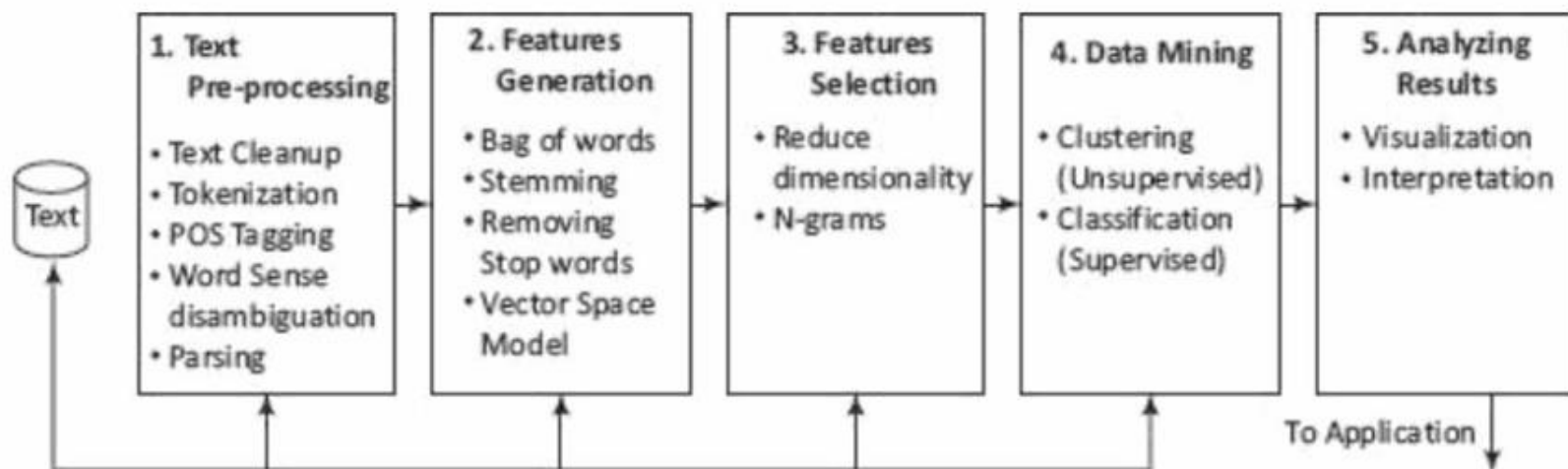
IE requires a dictionary of extraction patterns (For example, "Citizen of <x>, or "Located in <x>") and a semantic lexicon (dictionary of words with semantic category labels).

Document Clustering is an application which groups text documents into clusters. Automating document organization, topic extraction and fast information retrieval or filtering use the document clustering method. For example, web document clustering facilitates easy search by users.

Document Classification is an application to classify text documents into classes or categories. The application is useful for publishers, news sites, blogs or areas where lot of contents are present.

Web Mining is an application of data mining techniques. They discover patterns from the web Data Store. The patterns facilitate understanding. They improve the services of web-based applications. Data mining of web usage provides the browsing behavior of a website.

Concept Extraction is an application that deals with the extraction of concept from textual data. Concept extraction is an area of text classification in which words and phrases are classified into a semantically similar group.



! WEB MINING, WEB CONTENT AND WEB USAGE ANALYTICS

Web is a collection of interrelated files at web servers. Web data refers to

(i) **web content**—text, image and records, (ii) **web structure**—hyperlinks and tags, and (iii) **web usage**—http logs and application server logs.

Features of web data are:

1. Volume of information and its ready availability
2. Heterogeneity
3. Variety and diversity (Information on almost every topic is available using different forms, such as text, structured tables and lists, images, audio and video.)
4. Mostly semi-structured due to the nested structure of HTML code
5. Hyperlinks among pages within a website, and across different websites
6. Redundant or similar information may be present in several pages
7. Mostly, the web page has multiple sections (divisions), such as main contents of the page, advertisements, navigation panels, common menu for all the pages of a website and copyright notices
8. A web form or HTML form on a web page enables a user to enter data that is sent to a server for processing
9. Website contents are dynamic in nature where information on the web pages constantly changes, and fast information growth takes place such as conversations between users, social media, etc.

Web Mining

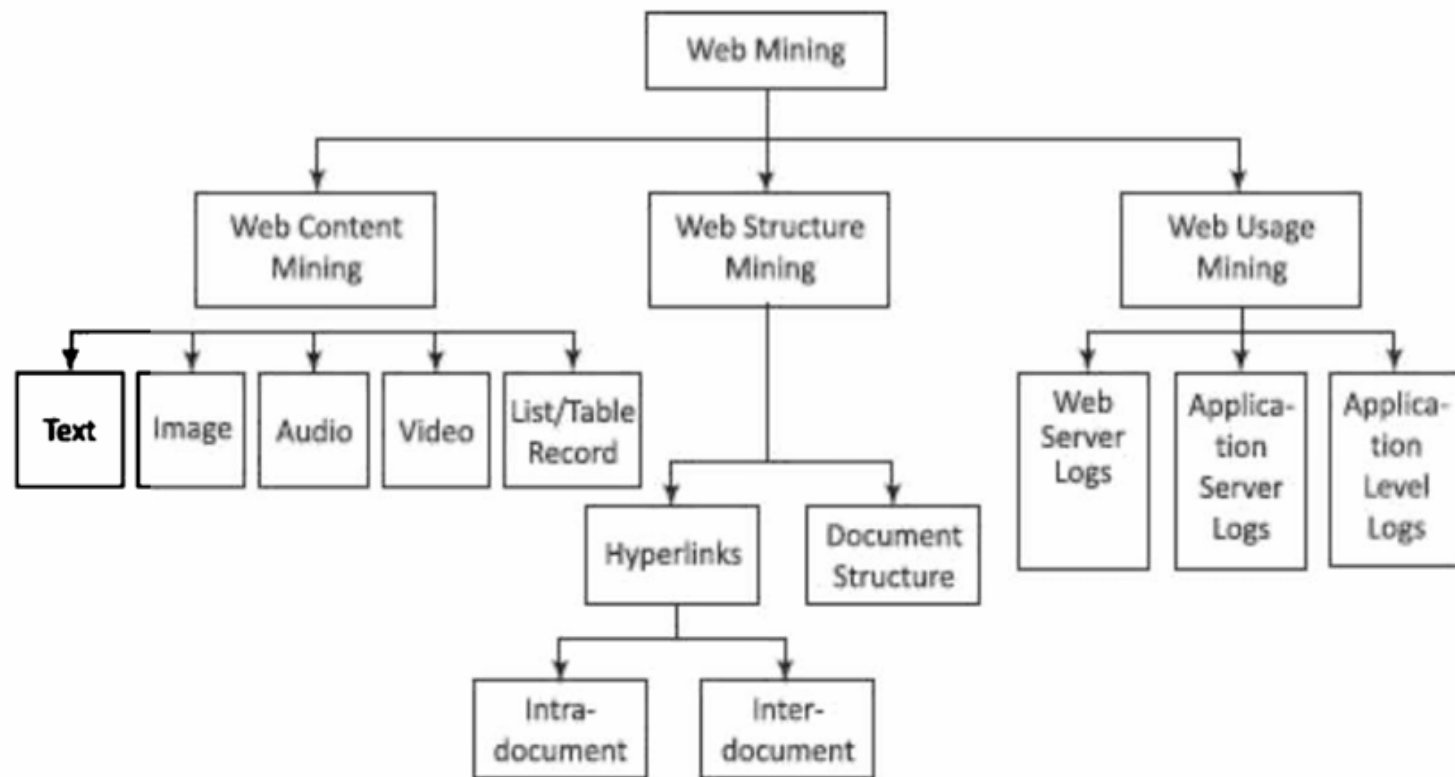
Web data mining is the mining of web data. Web mining methods are in multidisciplinary domains: (i) data mining, ML, natural language, (ii) processing, statistics, databases, information retrieval, and (iii) multimedia and visualization.

Web consists of rich features and patterns. A challenging task is retrieving interesting content and discovering knowledge from web data. Web offers several opportunities and challenges to data mining.

Definition of Web Mining

Web mining refers to the use of techniques and algorithms that extract knowledge from the web data available in the form of web documents and services. Web mining applications are as follows:

- (i) Extracting the fragment from a web document that represents the full web document
- (ii) Identifying interesting graph patterns or pre-processing the whole web graph to come up with metrics, such as PageRank
- (iii) User identification, session creation, malicious activity detection and filtering, and extracting usage path patterns



Web Content Mining

Web Content Mining is the process of **information** or **resource discovery** from the content of web documents across the World Wide Web. Web content mining can be (i) **direct mining** of the contents of documents or (ii) **mining through search engines**. They search fast compared to direct method.

Web content mining relates to both, data mining as well as text mining. Following are the reasons:

- (i) The content from web is similar to the contents obtained from database, file system or through any other mean. Thus available data mining techniques can be applied to the web.
- (ii) Content mining relates to text mining because much of the web content comprises texts.
- (iii) Web data are mainly semi-structured and/or unstructured, while data mining is structured and the text is unstructured.

Applications

Following are the applications of content mining from web documents:

1. **Classifying the web documents into categories**
2. **Identifying topics of web documents**
3. **Finding similar web pages across the different web servers**
4. **Applications related to relevance:**
 - (a) Recommendations – List of top “n” relevant documents in a collection or portion of a collection
 - (b) Filters – Show/Hide documents based on some criterion
 - (c) Queries – Enhance standard query relevance with user, role, and/or task-based relevance.

9.3.2.1 Common Web Content Mining Techniques

Pre-processing of contents The pre-processing steps are quite similar to the pre-processing for text mining. The content preparation involves:

1. **Extraction of text from HTML**
2. **Data cleaning** by filling up the missing values and smoothing the noisy data
3. **Tokenizing**: Generates the tokens of words from the cleaned up text
4. **Stemming**: Reduce the words to their roots. The different grammatical forms or declinations of verbs identify and index (count) as the same word. For example, stemming will ensure that both “closed” and “closing” are derived from the same word “close”. Stemming algorithm, Porter, can be used here. The java code for Porter stemming algorithm can be obtained from <https://tartarus.org/martin/PorterStemmer/java.txt>,

5. *Removing the stop words:* The common words unlikely to help in the mining process such as articles (a, an, the), or prepositions (such as, to, in, for) are removed.
6. *Calculate collection wide-word frequencies:* The distinct-word stem obtained after stemming process and removing the stop words results into a list of significant words (or terms). Calculating the occurrence of a significant term (t) in a collection is called collection frequency (CF_t). CF counts the multiple occurrences.)

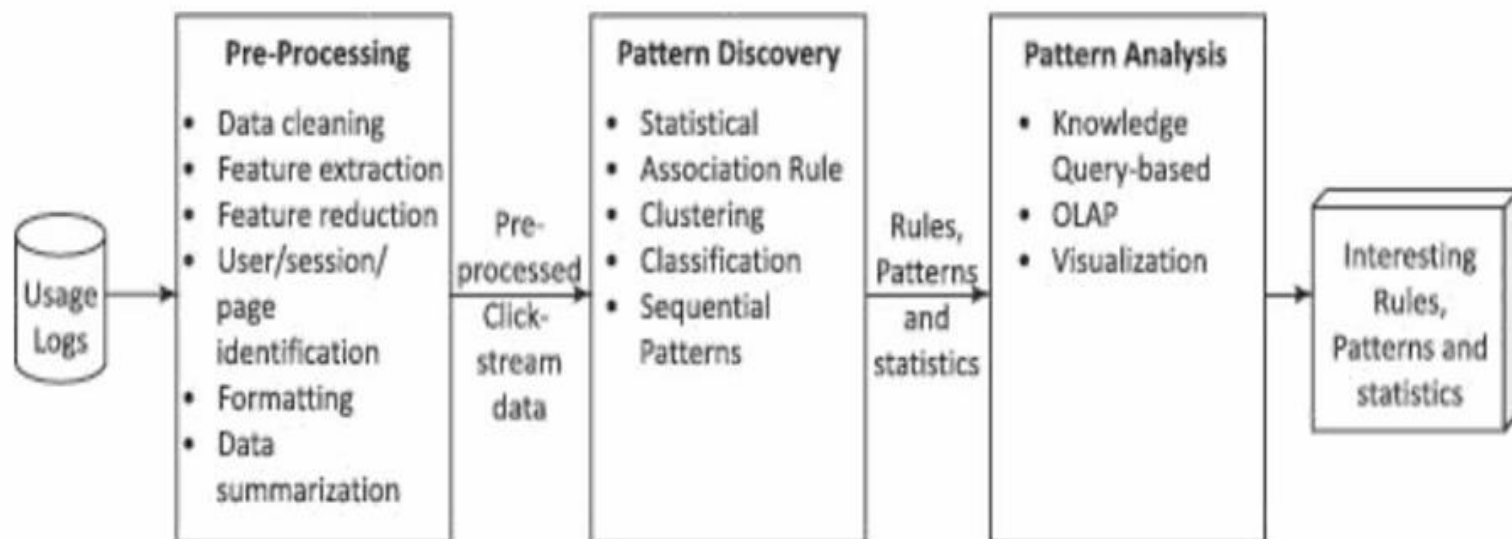
Now, find the number of documents in the collection that contains the specific term (t). This numeric measure is the document frequency (DF_t).

7. *Calculate per Document Term Frequencies (TF).* TF is a numeric measure that is used to score the importance of a word in a document based on how often it appeared in that document (Refer Example 9.1).
8. *Bag of words:* Web document is represented by the words it contains (and their occurrences).

Mining Tasks for Web Content Analytics

Following are the tasks for web content analytics:

1. **Classification** – A supervised technique which:
 - (i) Identifies the class or category a new web documents belongs to from the set of predefined classes or categories
 - (ii) Categories in the form of a term vector that are produced during a “training” phase
 - (iii) Employs algorithms using term vector to categorize the new data according to the observations at the training set.
2. **Clustering** – An unsupervised technique:
 - (i) Groups the web documents (clustered) with similar features using some similarity measure
 - (ii) Uses no pre-defined perception of what the groups should be
 - (iii) Measures most common similarity using the dot product between two web document vectors.
3. *Identifying the association* between web documents – Association rules help to identify correlation between web pages that occur mostly together.



9.3.3.2 Pattern Discovery

The pre-processed data enable the application of knowledge extraction algorithms based on statistics, ML and data mining algorithms. Mining algorithms, such as path analysis, association rules, sequential patterns, clustering and classification enable effective processing of web usages. The choice of mining techniques depends on the requirement of the analyst. Pre-processed data of the web access logs transform into knowledge to uncover the potential patterns and are further provided to pattern analysis phase.

Some of the techniques used for pattern discovery of web usage mining are:

Statistical techniques They are the most common methods which extract the knowledge about users. They perform different kinds of descriptive statistical analysis (frequency, mean, median) on variables such as page views, viewing time and length of path for navigational.

Statistical techniques enable discovering:

- (i) The most frequently accessed pages
- (ii) Average view time of a page or average length of a path through a site
- (iii) Providing support for marketing decisions

Association rule The rules enable relating the pages, which are most often referenced together in a single server session. These pages may not be directly connected to one another using the hyperlinks.

Other uses of association rule mining are:

- (i) **Reveal a correlation between users who visited a page containing** similar information. For example, a user visited a web page related to admission in an undergraduate course to those who search an eBook related to any subject.
- (ii) **Provide recommendations to purchase other products.** For example, recommend to user who visited a web page related to a book on data analytics, the books on ML and Big Data analytics also.

9.3.3.3 *Pattern Analysis*

The objective of pattern analysis is to filter out uninteresting rules or patterns from the rules, patterns or statistics obtained in the pattern discovery phase.

The most common form of pattern analysis consists of:

- (i) A knowledge query mechanism such as SQL
- (ii) Another method is to load usage data into a data cube in order to perform Online Analytical Processing (OLAP) operations
- (iii) Visualization techniques, such as graphing patterns or assigning the colors to different values, can often highlight overall patterns or trends in the data
- (iv) Content and structure information can filter out patterns containing pages of a certain usage type, content type or pages that match a certain hyperlink structure.

Data cube enables visualizing data from different angles. For example, toys data visualization using category, colour and children preferences. Another example, news from category, such as sports, success stories, films or targeted readers (children, college students, etc).

PAGE RANK, STRUCTURE OF WEB AND ANALYZING A WEB GRAPH

. Link analysis finds the answers to the following:

1. Can a linked (web) page rank them higher or lower?
2. Can the links be modeled as edges of graphs, structure of web as graph network, and applied the tools same as for graph analytics?
3. Can web graph mining method analyze and find a link sending spams?
4. Does a set of links correspond to a hub? Do the links correspond to an authority?
5. Does a linked page has higher authority compared to others?

9.4.1 Page Rank Definition

The in-degree (visibility) of a link is the measure of number of in-links from other links. The out-degree (luminosity) of a link is number of other links to which that link points.

PageRank definition according to earlier approaches

Assume a web structure of hyperlinks. Each hyperlink in-links to a number of hyperlinks and out-links to a number of pages. A page commanding higher authority (rank) has greater number of in-degrees than out-degrees. Therefore, one measure of a page authority can be in-degrees with respect to out-degrees.

PageRank refers to the authority of the page measured in terms of number of times a link is sought after.

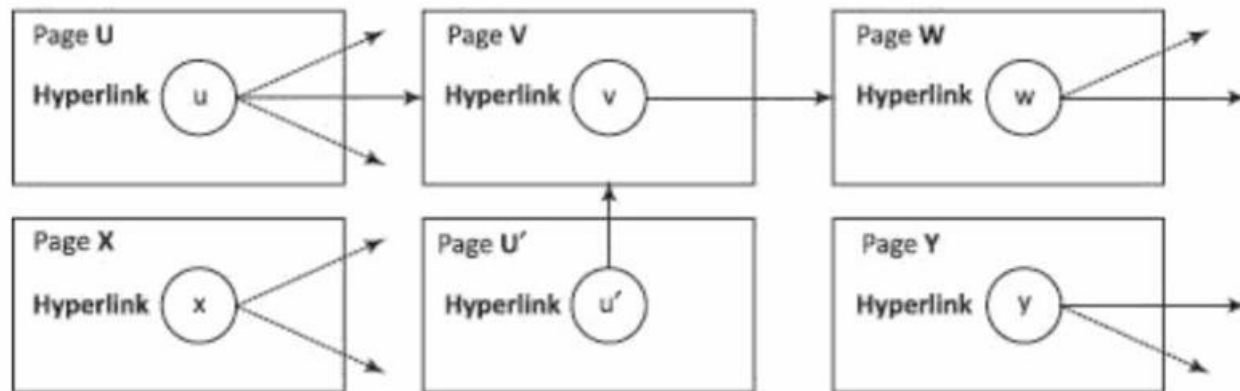
PageRank definition according to the new approach

Earlier approach of page ranking based on in-links and out-links does not capture the relative authority (importance) of the parents. Page and co-authors (1998) defined a page ranking method,⁵ which considers the entire web in place of local neighbourhood of the pages and considers the relative authority of the parent links (over children).

Web structure models as directed-graphs network-organization. Vertex of the directed graph models an anchor. Let $n = \text{number of hyperlinks at the page } U$. Assume $\mathbf{u}$ is a vector with elements $u_1, u_2, \dots, u_n$. Each page $Pg(\mathbf{u})$ has anchors, called hyperlinks. Page $Pg(\mathbf{v})$ consists of text document with m number of hyperlinks. $\mathbf{v}$ is a vector with elements $v_1, v_2, \dots, v_m$. The m is number of hyperlinks at $Pg(\mathbf{v})$. A vertex u directs to another Page V . A page $Pg(\mathbf{v})$ may have number of hyperlinks directed by out-edges to other page $Pg(\mathbf{w})$. Consider the following hypotheses:

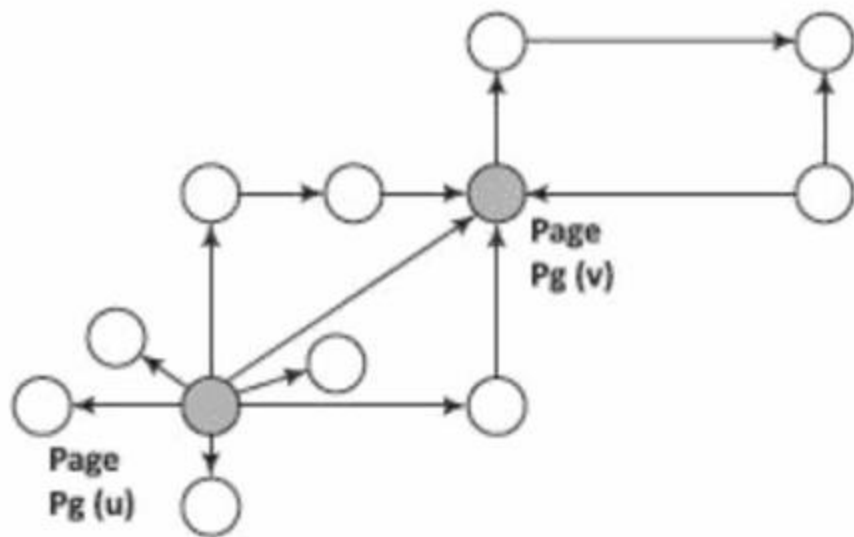
1. Text at the hyperlink represents the property of a vertex u that describes the destination V of the out-going edge.
2. A hyperlink in-between the pages represents the conferring of the authority.

Pages U and U' hyperlinks u and u' out-linking to Page V . Let Page U has three hyperlinks parenting three Pages, V one, W two, X two, U' one, and Y two, respectively. Figure 9.8 shows a web structure consisting of pages and hyperlinks.



Web structure with hyperlinks from a parent to one or more pages

Computation of PageRank and PageRank Iteration



Assume S corresponds to a set of pages. Initialize $\forall P_g \in S$. Symbols mean that initialize all pages P_g contained in the S and initialize Page Rank (PR_g) for each page as follows:

$$PR_{init}(P_g) = 1/|S| \quad (9.22)$$

How are the page ranks of the pages in a given set of pages iterated and computed till the ranks do not change (within specified margin, that means until converge)?

- S = Set of all pages in the system.
- $|S|$ = Total number of pages.
- $PR(P_g) = \text{PageRank of page } P_g$.
- $N(P_g) = \text{Number of outbound links from page } P_g$.

We initialize the PageRank of each page equally:

$$PR_{init}(P_g) = \frac{1}{|S|}$$

1. For each page $P_{gv} \in S$, compute:

$$PR_{\text{new}}(P_{gv}) = \sum_{P_{gu} \in B(P_{gv})} \frac{PR(P_{gu})}{N(P_{gu})}$$

3. **Normalization** (if needed):

Ensure that the sum of PageRanks across all pages is 1. You can normalize by:

$$\pi = \sum_{P_{gv} \in S} PR_{\text{new}}(P_{gv})$$

$$\text{Then for each } P_{gv} : \quad PR_{\text{new}}(P_{gv}) = \frac{PR_{\text{new}}(P_{gv})}{\pi}$$

